

Exploiting Stack Exchange as a Corpus Source for Technical Question Difficulty

A methodological report on corpus construction, validation, and reproducible data design

Arman Karapetyan
Université de Toulouse
CLLE (UMR 5263), Université Toulouse – Jean Jaurès, CNRS

May 11, 2026

Contents

1	Executive Summary	2
2	Source Verification And Scope	2
3	Introduction	2
4	Platform Background	2
4.1	Public Objects	3
4.2	Public Schema	3
5	Corpus Evidence	4
6	Access Methods	4
6.1	Stack Exchange Data Explorer	5
6.2	Public Stack Exchange Data Dump	5
6.3	Stack Exchange API	5
6.4	HTML Scraping	6
7	Corpus Construction Method	6
7.1	Site And Unit Selection	6
7.2	Sampling Design	6
7.3	Raw Storage And Processing	6
7.4	Derived Indicators	7
7.5	Provenance Record	7
8	Corpus Design	8
8.1	Field Groups	8
8.2	Minimal Storage Model	8
8.3	JSONL View	9
9	Validation And Completion	9
9.1	Verification Checks	9
9.2	Completion Actions	10
10	Licensing And Provenance	10

11 Relation To The State-Of-The-Art Review	10
12 Limitations	11
13 Conclusion And Recommendations	11
14 References	11

1 Executive Summary

Stack Exchange is a viable corpus source for technical question difficulty and answerability when records are thread-centered and traceable to official platform fields. This design addresses four requirements: platform description, technical construction method, conceptual corpus design, and quality-control procedures.

Thread-level records keep the platform context needed to study formulation, answerability, community response, timing, and provenance¹.

The recommended sequence starts with a Stack Overflow pilot of 5,000-10,000 questions using [Stack Exchange Data Explorer \(SEDE\)](#). If the pilot validates the design, the target corpus should contain 100,000-500,000 questions from the [public Stack Exchange Data Dump](#). The [Stack Exchange API](#) is reserved for enrichment, and HTML scraping is excluded by default.

2 Source Verification And Scope

Platform, access-method, schema, and licensing facts were verified from official Stack Exchange sources on 2026-05-11. Because access rules and platform documentation can change, any later extraction must record a new access date.

The present document is a protocol only: it does not include dataset extraction, API calls, SEDE exports, Data Dump downloads, model evaluation, or benchmark release.

3 Introduction

The feasibility question is whether public technical Q&A threads can support a reproducible corpus for later empirical work on question difficulty, answerability, and human/model performance.

Stack Exchange contains authentic technical questions written by real users and answered under public community norms. Stack Overflow is the initial site because it is large, programming-focused, code-rich, and directly related to software-engineering problems. Other Stack Exchange communities may support transfer analysis after the basic design is validated.

A usable corpus must therefore preserve the context in which a question was asked and answered, keep derived indicators traceable to original platform fields, and document access, transformation, licensing, and source dates so another researcher can reproduce the extraction.

4 Platform Background

Stack Exchange is a network of specialized question-answering communities. Each site is organized around a topic area and has its own tag vocabulary, moderation norms, accepted

¹Provenance means the recorded origin and processing history of a record, including source, access date, license, transformation steps, and hashes.

question types, and community expectations. A post that is acceptable on one site may be closed or migrated on another. These norms matter because closure, answer count, score, and latency are partly products of the community setting.

The research unit for the corpus is the question-centered discussion thread. A thread starts with a question and can contain answers, comments, edits, tags, scores, accepted-answer information, closure status, duplicate links, and timestamps. This unit preserves more difficulty and answerability evidence than an isolated question-answer pair. Clarifying comments may reveal missing context, duplicate links may reveal reformulation opportunities, and edit history may show instability in the original formulation.

4.1 Public Objects

The main public objects map directly to corpus roles.

Object	Platform role	Corpus role
Question	Initial post describing a problem	Main text and thread anchor
Answer	Proposed solution or explanation	Human solution candidate
Comment	Short discussion attached to a post	Clarification, correction, missing context
Tag	Topic or technology label	Sampling stratum and topic control
Score/views	Community reception and visibility	Exposure and popularity controls
Accepted answer	Answer selected by the asker	Resolution signal requiring validation
Edit history	Changes to title, body, or tags	Formulation and revision evidence
Closure status	Duplicate, unclear, too broad, off topic, or similar outcome	Suitability and answerability signal
Duplicate link	Relation to an existing question	Related-question evidence
Timestamp	Creation, activity, answer, edit, or closure time	Latency and temporal control

Only public post content and public or aggregate metadata are used. Private data and unnecessary user profile fields are excluded, and user-level fields are minimized to identifiers required for attribution, schema joins, or a justified research question.

4.2 Public Schema

The corresponding [public schema for the Data Dump and SEDE](#) is documented on Meta Stack Exchange. The `Posts` table contains non-deleted posts. `PostTypeId = 1` indicates questions and `PostTypeId = 2` indicates answers. Core question fields include `AcceptedAnswerId`, `CreationDate`, `Score`, `ViewCount`, `Body`, `Title`, `Tags`, `AnswerCount`, `CommentCount`, `ClosedDate`, and `ContentLicense`.

`Posts.Body` is rendered HTML². A plain-text body is therefore a derived field, not a replacement

²Rendered HTML is the display-ready HTML stored by Stack Exchange after the original post markup has been converted for presentation.

for the original content. `Comments` records include fields such as `PostId`, `Score`, `Text`, and `CreationDate`. `PostHistory` records edit and closure events; its body-edit entries expose raw markdown³ and are useful when formulation changes need to be studied. `PostLinks` records linked and duplicate relationships.

One version-specific processing rule is required: Data Dump versions from July 2025 onward must exclude the artificial post IDs 1000000001 and 1000000010 from `Posts.xml`.

5 Corpus Evidence

The same platform objects become useful research evidence only when grouped by analytical role. This prevents the dataset from becoming a flat collection of text documents.

Evidence category	Fields or objects	Use in the corpus
Question content	title, body, tags, code blocks, links, error messages	Problem formulation and topic control
Answer traces	answer bodies, scores, accepted flag, answer count	Human solution candidates and answerability evidence
Interaction traces	comments, edits, comment timing	Clarification, missing-context, correction, and instability signals
Moderation traces	closure status, duplicate links, migrations when available	Suitability, duplication, and formulation evidence
Timing traces	creation date, first answer time, accepted answer time, last activity	Latency, temporal validity, and response effort
Provenance traces	source URL, access method, source version, license, hash	Reproducibility and attribution

These fields require controlled interpretation across visibility, site norms, tag traffic, moderation, and time; Section 12 discusses those validity limits.

The design also separates early evidence from later community outcomes. Early evidence is available at question creation: title, body, tags, code, error messages, and creation date. Late evidence appears after community interaction: answers, comments, accepted-answer status, scores, closure, duplicates, edits, and latency. Keeping the two layers separate prevents later outcomes from entering early prediction or sampling variables.

6 Access Methods

Access is organized by role: pilot exploration, large-corpus extraction, targeted enrichment, and exceptional page-presentation use.

Method	Main role	Strength	Limitation	Recommended use
Stack Exchange Data Explorer (SEDE)	SQL access to public snapshots	Fast schema exploration and pilot export	Periodic refresh affects reproducibility	First pilot

³Raw markdown is the original lightweight markup written by a user before it is rendered into HTML.

Method	Main role	Strength	Limitation	Recommended use
Public Stack Exchange Data Dump	Large public files by site	Best source for large reproducible corpus	Requires parsing, storage, and versioning	Target corpus
Stack Exchange API	JSON access to current objects	Targeted checks and enrichment	Quotas, paging, filters, caching, throttles	Enrichment only
HTML scraping	Page presentation	Can expose display details	Fragile and unnecessary for required fields	Excluded by default

6.1 Stack Exchange Data Explorer

[SEDE](#)⁴ is an official web interface for running SQL queries against public Stack Exchange network data. It supports schema inspection, public query writing, query parameters, community queries, and CSV-style export. In the 5,000-10,000 question pilot, it enables rapid testing of fields, joins, and sampling strata before any Data Dump parser is implemented.

A SEDE pilot must save the SQL query text, target site database, query execution date, export file name, raw export hash, SEDE data refresh date if visible, and a short description of filters. Because snapshots are refreshed, SEDE is not the final source for a stable large corpus; it is the most efficient way to validate feasibility.

6.2 Public Stack Exchange Data Dump

At target scale, the [public Data Dump](#)⁵ is the preferred source. Current Stack Overflow help documentation describes data dumps as quarterly files made available for each Stack Exchange site through account settings.

The Data Dump requires engineering work: download, versioning, XML parsing, table linking, raw-file preservation, processed-file generation, and hash documentation. This work is justified only after the SEDE pilot validates field availability and sampling design. The parser must also apply the artificial-post exclusion rule described in Section 4.2 for Data Dump versions from July 2025 onward.

6.3 Stack Exchange API

The [Stack Exchange API](#)⁶ documentation checked on 2026-05-11 is for API v2.3. Responses use JSON in a common wrapper and support access to current Stack Exchange objects. Within this protocol, API use is limited to verifying specific records, retrieving current linked objects, or filling targeted fields missing from a pilot sample.

API use must document endpoint names, query parameters, [custom filters](#), [paging](#), throttles, caching, quotas, request dates, and attribution. Paging starts at page 1, defaults to pagesize 30, and allows pagesize values up to 100. The [throttle documentation](#) states that more than 30

⁴SEDE, or Stack Exchange Data Explorer, is the official web interface for running SQL queries against public Stack Exchange data snapshots.

⁵The public Data Dump is a periodic downloadable export of public Stack Exchange site data, distributed as site-level files rather than interactive query results.

⁶An API, or application programming interface, provides structured programmatic access to current Stack Exchange objects through documented web endpoints.

requests per second from one IP is abusive. The default daily quota is 10,000. Any `backoff`⁷ value returned by the API must be respected. The general API documentation also states that semantically identical polling faster than once per minute is abusive.

Large-scale crawling through the API is less reproducible and more constrained than processing the Data Dump, so it is not the main extraction method.

6.4 HTML Scraping

Direct HTML scraping⁸ is not part of the default protocol. It is fragile, harder to reproduce, and unnecessary when official methods provide the required corpus fields. It remains only as an exceptional, permission-based option for page-presentation details unavailable through SEDE, the Data Dump, or the API.

7 Corpus Construction Method

Construction proceeds from a controlled pilot to a target corpus. Scaling occurs only after the pilot confirms field availability, sampling coverage, and record quality.

7.1 Site And Unit Selection

Stack Overflow is the first site because it provides programming questions, code snippets, error messages, accepted answers, technology tags, duplicate links, closure decisions, and extensive answer/comment activity. Including other Stack Exchange sites at the start would mix site norms, moderation policies, topic distributions, and answer styles before the construction protocol is stable.

At record level, `question_id` links source fields, derived indicators⁹, validation fields, and provenance.

7.2 Sampling Design

The SEDE pilot contains 5,000-10,000 Stack Overflow questions. After validation, the design scales to 100,000-500,000 questions from the Data Dump.

Sampling is stratified rather than purely random. Answer status, accepted-answer status, closure status, duplicate status, tag popularity, and time period provide the pilot strata. The objective is enough variation to evaluate answerability, difficulty-related traces, and corpus suitability.

Time-period strata also support temporal analysis: older and newer threads may differ in platform norms, technology versions, documentation quality, and potential exposure in model-training data.

7.3 Raw Storage And Processing

Raw exports are preserved without manual editing, for example under:

`data/raw/stackexchange-difficulty/`

Documented scripts generate processed data from those raw files, for example under:

⁷A backoff value is an API response instruction telling the client how long to wait before making the next request.

⁸HTML scraping means extracting data from rendered web pages rather than using official structured exports or documented API endpoints.

⁹Derived indicators are variables computed from original platform fields, such as answerability status or time to first answer.

data/processed/stackexchange-difficulty/

The default processed formats are relational-style tables for joins and JSONL¹⁰ records for thread-level analysis. Processed data do not replace raw data. When the Data Dump is used, rendered HTML from `Posts.Body` is preserved and plain text is produced only as a derived field. When `PostHistory` is used, raw markdown revisions remain separate from rendered current post bodies.

For API enrichment, the stored JSON preserves the endpoint, filter name or identifier, page and pagesize parameters, `has_more` paging behavior, any `backoff` response, request date, and response hash when feasible.

7.4 Derived Indicators

Derived indicators are computed only from preserved source fields. The pilot uses four indicator groups:

Indicator group	Fields
Answerability status	<code>has_answer</code> , <code>has_accepted_answer</code> , <code>is_unanswered</code> , <code>is_closed</code> , <code>is_duplicate</code>
Timing and interaction	<code>time_to_first_answer_hours</code> , <code>time_to_accepted_answer_hours</code> , <code>comment_count_before_first_answer</code>
Topic and visibility controls	<code>tag_popularity_bucket</code> , <code>rare_tag_flag</code> , score, view count
Formulation features	<code>question_length</code> , <code>code_block_count</code> , <code>contains_error_message</code>

Every derived indicator must have a documented definition. For example, `is_unanswered` must specify whether it means zero answers, no accepted answer, or the platform's own unanswered definition, because these alternatives are not equivalent.

7.5 Provenance Record

Provenance is stored with the dataset material, for example:

reports/datasets/stackexchange-difficulty/provenance.yml

The record includes access method, source version, query or dump file, export date, official source access date, license, transformation steps, raw export hash, processed output hash, and script version if scripts are introduced. It also includes `official_source_checked_at` and `source_url_checked_at` fields, such as:

```
official_source_checked_at: "2026-05-11"
source_url_checked_at:
  stack_exchange_api_docs: "2026-05-11"
  stack_exchange_data_dump_help: "2026-05-11"
  stack_exchange_licensing_help: "2026-05-11"
  stack_exchange_schema_meta: "2026-05-11"
```

¹⁰JSONL, or JSON Lines, stores one JSON object per line, which is convenient for thread-level records that can be streamed or processed independently.

8 Corpus Design

At schema level, source fields, derived indicators, validation fields, and provenance remain separate.

8.1 Field Groups

Field group	Examples	Purpose
Question fields	id, title, body HTML, derived body text, tags, creation date, score, view count	Main problem formulation and topic evidence
Answer fields	answer id, parent question id, body HTML, score, creation date, accepted flag	Human solution candidates and resolution evidence
Comment fields	comment id, parent post id, text, score, creation date	Clarification and correction evidence
Link and status fields	closed date, close reason when available, duplicate link, related post ids	Moderation and related-question evidence
Timing fields	first answer time, accepted answer time, last activity	Latency and temporal controls
Derived fields	answerability status, formulation features, tag popularity bucket	Analysis-ready variables
Validation fields	manual inspection status, suitability flag, notes	Corpus quality control
Provenance fields	source URL, access method, version, license, export date, hash	Reproducibility and attribution

8.2 Minimal Storage Model

A minimal relational representation is sufficient for the pilot:

Table/file	Main key	Function
questions	question_id	One row per question
answers	answer_id	One row per answer
comments	comment_id	One row per included comment
post_links	post_link_id	Duplicate and linked-question relations
post_history	post_history_id	Edits, raw markdown revisions, closures, migrations when used
tags	tag_name or tag_id	Tag metadata and frequency
derived_thread_indicators	question_id	Computed indicators
validation_audit	question_id	Manual inspection and suitability notes

Table/file	Main key	Function
<code>manual_difficulty_labels</code>	<code>question_id</code>	Optional later easy/difficult annotations, rubric identifiers, agreement, and uncertainty notes
<code>future_empirical_runs</code>	<code>run_id</code>	Optional later model answers, validation status, evaluation notes, and cost metadata
<code>provenance</code>	<code>dataset_version</code>	Source, license, access method, transformations, hashes

The initial storage model includes questions, answers, tags, thread status, provenance, and a small derived-indicator table. Comments, post history, duplicate links, manual labels, and future empirical runs are added when validation requires them.

8.3 JSONL View

A JSONL view can then be generated from the relational files as an analysis export; the relational representation remains authoritative.

9 Validation And Completion

Validation checks both technical integrity and analytical usefulness. It combines row-level checks, distribution checks, and manual inspection.

9.1 Verification Checks

Check	Purpose
Row count	Confirm expected corpus size
Unique question IDs	Detect duplicates
Missing-value analysis	Identify unavailable answers, comments, tags, closure fields
Tag distribution	Detect overrepresentation of popular technologies
Answered/unanswered distribution	Check answerability balance
Accepted/no accepted distribution	Check resolution-signal balance
Closure distribution	Ensure closed and unsuitable cases are represented
Duplicate distribution	Check related-question coverage
Time-to-first-answer distribution	Inspect latency and interaction effort
Score/view distribution	Detect visibility bias
Manual inspection of at least 100 records	Verify readability, field linking, and suitability

In the manual sample, inspection verifies that question bodies are readable, code and error messages are preserved, answers and accepted answers are correctly linked, duplicate links are meaningful, comments add useful context, and record-level provenance is complete.

9.2 Completion Actions

Problem found	Completion action
Too many popular or easy questions	Oversample rare tags, unanswered questions, or long-latency questions
Too few answerable cases	Add accepted-answer and top-scored-answer threads
Too few difficult or ambiguous cases	Add closed, no-accepted-answer, unanswered, or high-comment threads
Too few recent technologies	Stratify by creation date or tag recency
Comments missing but needed	Use the Data Dump or targeted API enrichment
Accepted answers unreliable	Compare with top-scored answers and add manual validation
Corpus too narrow by tag	Add tag-stratified sampling
Records unsuitable for later evaluation	Mark as unsuitable instead of silently deleting
Provenance incomplete	Exclude or re-export with documented query/date/hash

The validation step ends with a corpus audit covering included records, missing fields, residual biases, and readiness for empirical comparison.

10 Licensing And Provenance

[Stack Overflow's licensing help page](#) states that publicly accessible user contributions are licensed under Creative Commons Attribution-ShareAlike¹¹, with the applicable version depending on contribution date:

- before 2011-04-08: CC BY-SA 2.5;
- from 2011-04-08 to before 2018-05-02: CC BY-SA 3.0;
- on or after 2018-05-02: CC BY-SA 4.0.

Attribution must be preserved according to the applicable Stack Exchange terms and Creative Commons license. Minimum provenance fields are source site, post id, answer ids, comment ids if used, canonical URL, access method, SEDE query or dump version, retrieval/export date, official source access date, license, transformation steps, raw export hash, processed output hash, and script version if scripts are later introduced.

Because access terms and interfaces can change, extraction must start with a fresh check of the Stack Exchange help pages, [Public Network Terms of Service](#), [API Terms of Use](#), [API documentation](#), [Data Dump help page](#), [licensing help page](#), and [schema documentation](#). The new access date belongs in the dataset provenance, not only in working notes.

11 Relation To The State-Of-The-Art Review

The companion [state-of-the-art review](#) defines the prediction framework for retrieval, prompting, community question difficulty, generated-answer evaluation, and human/model comparison. The present report supplies the corpus layer required by that framework: observable Stack Exchange

¹¹Creative Commons Attribution-ShareAlike, abbreviated CC BY-SA, allows reuse with attribution and requires adapted material to be shared under compatible terms.

records, source control, validation, and provenance. Predictors, decision policies, cost models, and utility functions belong to later empirical experiments.

12 Limitations

The main construct limitation follows from the observational nature of platform traces. Accepted answer status is a resolution signal. No-answer status can reflect difficulty, low visibility, asker abandonment, or topic scarcity. Score reflects reception and visibility as well as quality. Closure may indicate moderation fit rather than intrinsic difficulty.

Internal validity depends on the early/late evidence split defined in Section 5; later community outcomes can leak outcome information into early prediction tasks.

External validity is limited by the first-site choice. Stack Overflow is large and technically relevant, but it has specific norms about reproducibility, code examples, acceptable questions, and moderation. A corpus built only from highly active tags may not generalize to rare technologies or other Stack Exchange communities.

Temporal validity matters because answers may become obsolete, APIs and libraries change, and community norms evolve. Question difficulty may change as documentation, tooling, and model capabilities improve.

Later model evaluation adds training-contamination, correctness, prompt-variant, retrieval, and execution-environment risks. A model may have seen Stack Overflow content during training. Fluent answers can be wrong. Correctness requires validation beyond style or preference.

Undocumented filtering, missing attribution, or missing source/version metadata weakens reproducibility.

13 Conclusion And Recommendations

Before empirical comparison begins, the corpus audit should be complete and raw exports, processed transformations, rendered HTML, derived text, and `PostHistory` markdown should remain traceable.

The resulting corpus keeps answerability, human difficulty annotations, formulation features, timing traces, and model-evaluation outcomes separate.

14 References

All links in this section were checked on 2026-05-11.

- [Stack Exchange API documentation](#)
- [API filters](#)
- [API paging](#)
- [API throttles](#)
- [Stack Exchange Data Explorer help](#)
- [SEDE query tutorial](#)
- [Stack Overflow data dump help](#)
- [Stack Overflow licensing help](#)
- [Public Network Terms of Service](#)
- [API Terms of Use](#)
- [Public data dump and SEDE schema documentation](#)